Lines 1-84

```python
import datetime as dt
import os
import uuid

from flask import Flask, abort, flash, redirect, render_template, request, url_for
from flask_login import LoginManager, UserMixin, current_user, login_required, login_user, logout_user
from sqlite3 import IntegrityError
from werkzeug.security import check_password_hash, generate_password_hash
from werkzeug.utils import secure_filename

from db import get_db_connection


# Configuration

app = Flask(__name__)
app.secret_key = os.environ.get("SECRET_KEY", "dev-secret-key")

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
UPLOAD_FOLDER = os.path.join(BASE_DIR, "static", "uploads")
app.config["UPLOAD_FOLDER"] = UPLOAD_FOLDER
os.makedirs(UPLOAD_FOLDER, exist_ok=True)

ALLOWED_EXTENSIONS = {"png", "jpg", "jpeg", "gif", "webp"}
LANGUAGES = ["Italian", "English", "Spanish", "Portuguese", "German"]
WEEKDAYS = [
    (0, "Monday"),
    (1, "Tuesday"),
    (2, "Wednesday"),
    (3, "Thursday"),
    (4, "Friday"),
    (5, "Saturday"),
    (6, "Sunday"),
]
WEEKDAY_NAMES = dict(WEEKDAYS)
DURATION_FILTERS = {
    "short": ("Up to 90 min", 0, 90),
    "medium": ("91-150 min", 91, 150),
    "long": ("Over 150 min", 151, 10000),
}
MIN_TOUR_STOPS = 4

login_manager = LoginManager()
login_manager.init_app(app)
login_manager.login_view = "login"


# Authentication

class User(UserMixin):
    def __init__(self, id, email, role, first_name, last_name, languages=""):
        self.id = id
        self.email = email
        self.role = role
        self.first_name = first_name
        self.last_name = last_name
        self.languages = languages or ""

    @property
    def spoken_languages(self):
        return [lang.strip() for lang in self.languages.split(",") if lang.strip()]


@login_manager.user_loader
def load_user(user_id):
    conn = get_db_connection()
    user_row = conn.execute("SELECT * FROM users WHERE id = ?", (user_id,)).fetchone()
    conn.close()
    if user_row:
        return User(
            user_row["id"],
            user_row["email"],
            user_row["role"],
            user_row["first_name"],
            user_row["last_name"],
            user_row["languages"],
        )
    return None


@app.context_processor
def inject_constants():
    return {
        "LANGUAGES": LANGUAGES,
```

Lines 85-166

```python
        "WEEKDAYS": WEEKDAYS,
        "WEEKDAY_NAMES": WEEKDAY_NAMES,
        "DURATION_FILTERS": DURATION_FILTERS,
        "MIN_TOUR_STOPS": MIN_TOUR_STOPS,
    }


# Validation

def normalize_email(email):
    return (email or "").strip().lower()


def allowed_file(filename):
    return "." in filename and filename.rsplit(".", 1)[1].lower() in ALLOWED_EXTENSIONS


def validate_file(file):
    return file and file.filename and allowed_file(file.filename)


def save_uploaded_file(file, prefix):
    if not validate_file(file):
        raise ValueError("Invalid file. Use png, jpg, jpeg, gif or webp.")
    original = secure_filename(file.filename)
    ext = original.rsplit(".", 1)[1].lower()
    filename = f"{prefix}-{uuid.uuid4().hex}.{ext}"
    file.save(os.path.join(app.config["UPLOAD_FOLDER"], filename))
    return f"/static/uploads/{filename}"


def delete_uploaded_file(path):
    if not path or not path.startswith("/static/uploads/"):
        return
    upload_root = os.path.abspath(app.config["UPLOAD_FOLDER"])
    filename = secure_filename(os.path.basename(path))
    file_path = os.path.abspath(os.path.join(upload_root, filename))
    if os.path.commonpath([upload_root, file_path]) != upload_root:
        return
    if os.path.exists(file_path):
        os.remove(file_path)


def parse_positive_int(value, label, min_value=1, max_value=None):
    try:
        parsed = int(value)
    except (TypeError, ValueError):
        raise ValueError(f"{label} must be a number.")
    if parsed < min_value:
        raise ValueError(f"{label} must be at least {min_value}.")
    if max_value is not None and parsed > max_value:
        raise ValueError(f"{label} cannot be greater than {max_value}.")
    return parsed


def parse_time_to_minutes(value):
    try:
        parsed = dt.time.fromisoformat(value)
    except (TypeError, ValueError):
        raise ValueError("Invalid time.")
    return parsed.hour * 60 + parsed.minute


def minutes_to_time_label(total_minutes):
    hours = (total_minutes // 60) % 24
    minutes = total_minutes % 60
    return f"{hours:02d}:{minutes:02d}"


def time_range_label(start_time, duration_mins):
    start_minutes = parse_time_to_minutes(start_time)
    end_minutes = start_minutes + duration_mins
    return f"{minutes_to_time_label(start_minutes)} to {minutes_to_time_label(end_minutes)}"


def ranges_overlap(start_a, duration_a, start_b, duration_b):
    end_a = start_a + duration_a
    end_b = start_b + duration_b
    return start_a < end_b and start_b < end_a


def parse_iso_date(value):
```

Lines 167-246

```
    try:
        return dt.date.fromisoformat(value)
    except (TypeError, ValueError):
        raise ValueError("Invalid date.")


def tour_datetime(tour_date, start_time):
    return dt.datetime.combine(tour_date, dt.time.fromisoformat(start_time))


def split_names(value):
    raw = (value or "").replace("\n", ",")
    return [name.strip() for name in raw.split(",") if name.strip()]


def has_first_and_last_name(value):
    return len([part for part in value.split() if part.strip()]) >= 2


def safe_redirect(default_endpoint="index"):
    next_url = request.args.get("next") or request.form.get("next")
    if next_url and next_url.startswith("/") and not next_url.startswith("//"):
        return redirect(next_url)
    return redirect(url_for(default_endpoint))


def require_role(role):
    if not current_user.is_authenticated:
        return redirect(url_for("login", next=request.path))
    if current_user.role != role:
        flash("This account does not have permission to perform this action.", "warning")
        return redirect(url_for("index"))
    return None


# Tour data

def get_schedules(conn, tour_id):
    return conn.execute(
        "SELECT * FROM tour_schedule WHERE tour_id = ? ORDER BY weekday",
        (tour_id,),
    ).fetchall()


def get_schedule_for_date(conn, tour_id, tour_date):
    return conn.execute(
        """
        SELECT * FROM tour_schedule
        WHERE tour_id = ? AND weekday = ?
        """,
        (tour_id, tour_date.weekday()),
    ).fetchone()


def format_schedule(schedules):
    return ", ".join(f"{WEEKDAY_NAMES[row['weekday']]} {row['start_time']}" for row in schedules)


def get_stops(conn, tour_id):
    return conn.execute(
        "SELECT * FROM tour_stops WHERE tour_id = ? ORDER BY position",
        (tour_id,),
    ).fetchall()


def get_photos(conn, tour_id):
    return conn.execute(
        "SELECT * FROM tour_photos WHERE tour_id = ? ORDER BY position",
        (tour_id,),
    ).fetchall()


def primary_photo(conn, tour_id):
    photo = conn.execute(
        "SELECT path FROM tour_photos WHERE tour_id = ? ORDER BY position LIMIT 1",
        (tour_id,),
    ).fetchone()
    return photo["path"] if photo else "/static/assets/img/favicon.png"
```

Lines 247-330

```
def tour_like_count(conn, tour_id):
    row = conn.execute(
        "SELECT COUNT(*) AS total FROM tour_likes WHERE tour_id = ?",
        (tour_id,),
    ).fetchone()
    return row["total"]


def tour_comment_count(conn, tour_id):
    row = conn.execute(
        "SELECT COUNT(*) AS total FROM comments WHERE tour_id = ?",
        (tour_id,),
    ).fetchone()
    return row["total"]


def current_user_liked(conn, tour_id):
    if not current_user.is_authenticated:
        return False
    row = conn.execute(
        "SELECT 1 FROM tour_likes WHERE tour_id = ? AND user_id = ?",
        (tour_id, current_user.id),
    ).fetchone()
    return row is not None


def get_tour_row(conn, tour_id):
    return conn.execute(
        """
        SELECT tours.*, users.first_name AS guide_first_name,
               users.last_name AS guide_last_name
        FROM tours
        JOIN users ON users.id = tours.guide_id
        WHERE tours.id = ?
        """,
        (tour_id,),
    ).fetchone()


# Availability

def active_reserved_places(conn, tour_id, tour_date):
    row = conn.execute(
        """
        SELECT COALESCE(SUM(num_people), 0) AS total
        FROM reservations
        WHERE tour_id = ? AND tour_date = ? AND status = 'booked'
        """,
        (tour_id, tour_date.isoformat()),
    ).fetchone()
    return row["total"]


def available_places(conn, tour, tour_date):
    return tour["max_participants"] - active_reserved_places(conn, tour["id"], tour_date)


def tour_has_active_reservations(conn, tour_id):
    row = conn.execute(
        """
        SELECT COUNT(*) AS total
        FROM reservations
        WHERE tour_id = ? AND status = 'booked'
        """,
        (tour_id,),
    ).fetchone()
    return row["total"] > 0


def guide_has_overlap(conn, guide_id, weekday, start_time, duration_mins, exclude_tour_id=None):
    sql = """
        SELECT tours.id, tours.title, tours.duration_mins, tour_schedule.start_time
        FROM tours
        JOIN tour_schedule ON tour_schedule.tour_id = tours.id
        WHERE tours.guide_id = ? AND tour_schedule.weekday = ?
    """
    params = [guide_id, weekday]
    if exclude_tour_id is not None:
        sql += " AND tours.id <> ?"
        params.append(exclude_tour_id)

    start_minutes = parse_time_to_minutes(start_time)
    for row in conn.execute(sql, params).fetchall():
        other_start = parse_time_to_minutes(row["start_time"])
```

Lines 331-412

```
        if ranges_overlap(start_minutes, duration_mins, other_start, row["duration_mins"]):
            return row
    return None


def participant_has_overlap(conn, user_id, tour_date, start_time, duration_mins):
    rows = conn.execute(
        """
        SELECT reservations.id, tours.title, tours.duration_mins, tour_schedule.start_time
        FROM reservations
        JOIN tours ON tours.id = reservations.tour_id
        JOIN tour_schedule
             ON tour_schedule.tour_id = tours.id
            AND tour_schedule.weekday = ?
        WHERE reservations.user_id = ?
          AND reservations.tour_date = ?
          AND reservations.status = 'booked'
        """,
        (tour_date.weekday(), user_id, tour_date.isoformat()),
    ).fetchall()
    start_minutes = parse_time_to_minutes(start_time)
    for row in rows:
        other_start = parse_time_to_minutes(row["start_time"])
        if ranges_overlap(start_minutes, duration_mins, other_start, row["duration_mins"]):
            return row
    return None


def upcoming_dates(conn, tour, horizon_days=60):
    today = dt.date.today()
    now = dt.datetime.now()
    schedules = {row["weekday"]: row for row in get_schedules(conn, tour["id"])}
    dates = []
    for offset in range(horizon_days + 1):
        candidate = today + dt.timedelta(days=offset)
        schedule = schedules.get(candidate.weekday())
        if not schedule:
            continue
        start_dt = tour_datetime(candidate, schedule["start_time"])
        if start_dt <= now:
            continue
        seats_left = available_places(conn, tour, candidate)
        dates.append(
            {
                "date": candidate,
                "date_iso": candidate.isoformat(),
                "weekday": WEEKDAY_NAMES[candidate.weekday()],
                "start_time": schedule["start_time"],
                "seats_left": seats_left,
            }
        )
    return dates


def enrich_tour(conn, row, selected_date=None):
    tour = dict(row)
    tour["schedule"] = get_schedules(conn, tour["id"])
    tour["schedule_label"] = format_schedule(tour["schedule"])
    tour["primary_photo"] = primary_photo(conn, tour["id"])
    tour["can_edit"] = not tour_has_active_reservations(conn, tour["id"])
    tour["like_count"] = tour_like_count(conn, tour["id"])
    tour["comment_count"] = tour_comment_count(conn, tour["id"])
    tour["is_liked"] = current_user_liked(conn, tour["id"])
    if selected_date:
        schedule = get_schedule_for_date(conn, tour["id"], selected_date)
        tour["selected_seats_left"] = available_places(conn, tour, selected_date) if schedule else None
    return tour


def filtered_tours(conn, filters):
    rows = conn.execute(
        """
        SELECT tours.*, users.first_name AS guide_first_name,
               users.last_name AS guide_last_name
        FROM tours
        JOIN users ON users.id = tours.guide_id
        ORDER BY tours.created_at DESC, tours.id DESC
        """
    ).fetchall()

    selected_date = None
    if filters.get("date"):
```

Lines 413-490

```
        selected_date = parse_iso_date(filters["date"])

    results = []
    for row in rows:
        tour = enrich_tour(conn, row, selected_date)
        q = (filters.get("q") or "").strip().lower()
        if q and q not in tour["title"].lower() and q not in tour["theme"].lower():
            continue
        if filters.get("language") and tour["language"] != filters["language"]:
            continue
        duration_key = filters.get("duration")
        if duration_key:
            _, min_duration, max_duration = DURATION_FILTERS[duration_key]
            if not (min_duration <= tour["duration_mins"] <= max_duration):
                continue
        if selected_date and not get_schedule_for_date(conn, tour["id"], selected_date):
            continue
        results.append(tour)
    return results


# Tour forms

def parse_schedule_form():
    schedules = []
    errors = []
    for weekday, label in WEEKDAYS:
        if request.form.get(f"day_{weekday}"):
            start_time = (request.form.get(f"time_{weekday}") or "").strip()
            if not start_time:
                errors.append(f"Enter a time for {label}.")
                continue
            try:
                parse_time_to_minutes(start_time)
            except ValueError:
                errors.append(f"Invalid time for {label}.")
                continue
            schedules.append({"weekday": weekday, "start_time": start_time})
    if not schedules:
        errors.append("Select at least one weekday.")
    return schedules, errors


def parse_tour_form(conn, exclude_tour_id=None):
    errors = []
    title = (request.form.get("title") or "").strip()
    theme = (request.form.get("theme") or "").strip()
    meeting_point = (request.form.get("meeting_point") or "").strip()
    description = (request.form.get("description") or "").strip()
    language = (request.form.get("language") or "").strip()
    stops = split_names(request.form.get("stops"))

    if len(title) < 4:
        errors.append("The title must contain at least 4 characters.")
    if len(theme) < 3:
        errors.append("Enter a tour theme.")
    if len(meeting_point) < 4:
        errors.append("Enter a clear meeting point.")
    if len(description) < 30:
        errors.append("The description must contain at least 30 characters.")
    if language not in current_user.spoken_languages:
        errors.append("The tour language must be one of the guide's spoken languages.")
    if len(stops) < MIN_TOUR_STOPS:
        errors.append(f"Enter at least {MIN_TOUR_STOPS} stops.")

    try:
        duration_mins = parse_positive_int(request.form.get("duration_mins"), "Duration", 30, 360)
    except ValueError as exc:
        duration_mins = None
        errors.append(str(exc))

    try:
        max_participants = parse_positive_int(
            request.form.get("max_participants"),
            "Maximum number of participants",
            1,
            40,
        )
```

Lines 491-573

```
    except ValueError as exc:
        max_participants = None
        errors.append(str(exc))

    schedules, schedule_errors = parse_schedule_form()
    errors.extend(schedule_errors)

    if duration_mins:
        for schedule in schedules:
            overlap = guide_has_overlap(
                conn,
                current_user.id,
                schedule["weekday"],
                schedule["start_time"],
                duration_mins,
                exclude_tour_id,
            )
            if overlap:
                errors.append(
                    f"Schedule overlap: {WEEKDAY_NAMES[schedule['weekday']]} "
                    f"{schedule['start_time']} overlaps with '{overlap['title']}' "
                    f"from {time_range_label(overlap['start_time'], overlap['duration_mins'])}."
                )

    return {
        "title": title,
        "theme": theme,
        "meeting_point": meeting_point,
        "duration_mins": duration_mins,
        "language": language,
        "max_participants": max_participants,
        "description": description,
        "stops": stops,
        "schedules": schedules,
    }, errors


def get_form_photo_files():
    return [file for file in request.files.getlist("photos") if file and file.filename]


def validate_photo_files(files, required):
    if required and len(files) < 5:
        return "Upload at least 5 promotional photos."
    for file in files:
        if not validate_file(file):
            return "Photos must be png, jpg, jpeg, gif or webp."
    return None


def parse_photo_ids(values):
    photo_ids = set()
    for value in values:
        try:
            photo_id = int(value)
        except (TypeError, ValueError):
            raise ValueError("Invalid photo selection.")
        if photo_id <= 0:
            raise ValueError("Invalid photo selection.")
        photo_ids.add(photo_id)
    return photo_ids


def reindex_tour_photos(conn, tour_id):
    rows = conn.execute(
        "SELECT id FROM tour_photos WHERE tour_id = ? ORDER BY position, id",
        (tour_id,),
    ).fetchall()
    for position, row in enumerate(rows, start=1):
        conn.execute(
            "UPDATE tour_photos SET position = ? WHERE id = ?",
            (position, row["id"]),
        )


def insert_tour_details(conn, tour_id, data, photo_files):
    for schedule in data["schedules"]:
        conn.execute(
            "INSERT INTO tour_schedule (tour_id, weekday, start_time) VALUES (?, ?, ?)",
            (tour_id, schedule["weekday"], schedule["start_time"]),
        )
    for position, stop in enumerate(data["stops"], start=1):
        conn.execute(
```

Lines 574-655

```
            "INSERT INTO tour_stops (tour_id, name, position) VALUES (?, ?, ?)",
            (tour_id, stop, position),
        )
    for position, file in enumerate(photo_files, start=1):
        path = save_uploaded_file(file, f"tour-{tour_id}-{position}")
        conn.execute(
            "INSERT INTO tour_photos (tour_id, path, position) VALUES (?, ?, ?)",
            (tour_id, path, position),
        )


# Public pages

@app.route("/")
def index():
    conn = get_db_connection()
    tours = sorted(
        filtered_tours(conn, {}),
        key=lambda item: (item["like_count"], item["comment_count"], item["id"]),
        reverse=True,
    )[:3]
    conn.close()
    return render_template("index.html", tours=tours)


@app.route("/tours")
def tours():
    filters = {
        "q": request.args.get("q", ""),
        "date": request.args.get("date", ""),
        "duration": request.args.get("duration", ""),
        "language": request.args.get("language", ""),
    }
    if filters["duration"] and filters["duration"] not in DURATION_FILTERS:
        filters["duration"] = ""
    if filters["language"] and filters["language"] not in LANGUAGES:
        filters["language"] = ""

    conn = get_db_connection()
    try:
        tour_list = filtered_tours(conn, filters)
    except ValueError:
        flash("The date filter is not valid.", "warning")
        filters["date"] = ""
        tour_list = filtered_tours(conn, filters)
    conn.close()
    return render_template("tours.html", tours=tour_list, filters=filters)


# Account routes

@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        email = normalize_email(request.form.get("email"))
        password = request.form.get("password") or ""
        role = request.form.get("role")

        if role not in {"guide", "participant"}:
            flash("Select an account type.", "danger")
            return render_template("login.html")

        conn = get_db_connection()
        user_row = conn.execute(
            "SELECT * FROM users WHERE email = ? AND role = ?",
            (email, role),
        ).fetchone()
        conn.close()

        if user_row and check_password_hash(user_row["password"], password):
            user_obj = User(
                user_row["id"],
                user_row["email"],
                user_row["role"],
                user_row["first_name"],
                user_row["last_name"],
                user_row["languages"],
            )
            login_user(user_obj)
            return safe_redirect()
        flash("Wrong credentials for the selected account type.", "danger")
    return render_template("login.html")
```

Lines 656-737

```
656
657
658 @app.route("/register", methods=["GET", "POST"])
659 def register():
660     if request.method == "POST":
661         first_name = (request.form.get("first_name") or "").st
    > rip()
662         last_name = (request.form.get("last_name") or "").stri
    > p()
663         email = normalize_email(request.form.get("email"))
664         password = request.form.get("password") or ""
665         role = request.form.get("role")
666         selected_languages = request.form.getlist("languages")
667         errors = []
668
669         if len(first_name) < 2:
670             errors.append("Enter a valid first name.")
671         if len(last_name) < 2:
672             errors.append("Enter a valid last name.")
673         if "@" not in email or "." not in email:
674             errors.append("Enter a valid email.")
675         if len(password) < 6:
676             errors.append("The password must have at least 6 c
    > haracters.")
677         if role not in {"guide", "participant"}:
678             errors.append("Select a valid account type.")
679
680         languages = ""
681         if role == "guide":
682             valid_languages = [lang for lang in selected_langu
    > ages if lang in LANGUAGES]
683             if not valid_languages:
684                 errors.append("A guide must select at least on
    > e language.")
685             languages = ",".join(valid_languages)
686
687         if errors:
688             for error in errors:
689                 flash(error, "danger")
690             return render_template("register.html")
691
692         conn = get_db_connection()
693         try:
694             conn.execute(
695                 """
696                 INSERT INTO users (first_name, last_name, emai
    > l, password, role, languages)
697                 VALUES (?, ?, ?, ?, ?, ?)
698                 """,
699                 (
700                     first_name,
701                     last_name,
702                     email,
703                     generate_password_hash(password, method="p
    > bkdf2:sha256"),
704                     role,
705                     languages,
706                 ),
707             )
708             conn.commit()
709             flash("Registration completed. You can now sign in
    > .", "success")
710             return redirect(url_for("login"))
711         except IntegrityError:
712             flash("An account with this email already exists."
    > , "danger")
713         finally:
714             conn.close()
715     return render_template("register.html")
716
717
718 @app.route("/logout")
719 @login_required
720 def logout():
721     logout_user()
722     return redirect(url_for("index"))
723
724
725 # Guide tours
726
727 @app.route("/create-tour", methods=["GET", "POST"])
728 @login_required
729 def create_tour():
730     denied = require_role("guide")
731     if denied:
732         return denied
733
734     if request.method == "POST":
735         conn = get_db_connection()
736         data, errors = parse_tour_form(conn)
737         photo_files = get_form_photo_files()
```

Lines 738-816

```
738         photo_error = validate_photo_files(photo_files, requir
    > ed=True)
739         if photo_error:
740             errors.append(photo_error)
741
742         if errors:
743             for error in errors:
744                 flash(error, "danger")
745             conn.close()
746             return render_template("create_tour.html", tour=No
    > ne, selected_schedules={})
747
748         try:
749             cursor = conn.execute(
750                 """
751                 INSERT INTO tours
752                     (guide_id, title, theme, meeting_point, du
    > ration_mins,
753                      language, max_participants, description)
754                 VALUES (?, ?, ?, ?, ?, ?, ?, ?)
755                 """,
756                 (
757                     current_user.id,
758                     data["title"],
759                     data["theme"],
760                     data["meeting_point"],
761                     data["duration_mins"],
762                     data["language"],
763                     data["max_participants"],
764                     data["description"],
765                 ),
766             )
767             tour_id = cursor.lastrowid
768             insert_tour_details(conn, tour_id, data, photo_fil
    > es)
769             conn.commit()
770             flash("Tour planned successfully.", "success")
771             return redirect(url_for("tour_detail", id=tour_id)
    > )
772         except ValueError as exc:
773             conn.rollback()
774             flash(str(exc), "danger")
775         finally:
776             conn.close()
777
778     return render_template("create_tour.html", tour=None, sele
    > cted_schedules={})
779
780
781 @app.route("/tour/<int:id>/edit", methods=["GET", "POST"])
782 @login_required
783 def edit_tour(id):
784     denied = require_role("guide")
785     if denied:
786         return denied
787
788     conn = get_db_connection()
789     tour = get_tour_row(conn, id)
790     if not tour:
791         conn.close()
792         abort(404)
793     if tour["guide_id"] != int(current_user.id):
794         conn.close()
795         abort(403)
796     if tour_has_active_reservations(conn, id):
797         conn.close()
798         flash("This tour has active reservations and cannot be
    >  edited.", "warning")
799         return redirect(url_for("tour_detail", id=id))
800
801     selected_schedules = {row["weekday"]: row["start_time"] fo
    > r row in get_schedules(conn, id)}
802     stops = ", ".join(row["name"] for row in get_stops(conn, i
    > d))
803     current_photos = get_photos(conn, id)
804     tour_data = dict(tour)
805     tour_data["stops_text"] = stops
806
807     if request.method == "POST":
808         data, errors = parse_tour_form(conn, exclude_tour_id=i
    > d)
809         photo_files = get_form_photo_files()
810         remove_photo_ids = set()
811         photo_error = validate_photo_files(photo_files, requir
    > ed=False)
812         if photo_error:
813             errors.append(photo_error)
814         try:
815             remove_photo_ids = parse_photo_ids(request.form.ge
    > tlist("remove_photos"))
816             current_photo_ids = {photo["id"] for photo in curr
    > ent_photos}
```

Lines 817-892

```
            if remove_photo_ids - current_photo_ids:
                errors.append("Invalid photo selection.")
        except ValueError as exc:
            errors.append(str(exc))

        final_photo_count = len(current_photos) - len(remove_photo_ids) + len(photo_files)
        if (remove_photo_ids or photo_files) and final_photo_count < 5:
            errors.append("A tour must keep at least 5 promotional photos.")
        removed_photo_paths = [
            photo["path"] for photo in current_photos if photo["id"] in remove_photo_ids
        ]

        if errors:
            for error in errors:
                flash(error, "danger")
            conn.close()
            return render_template(
                "create_tour.html",
                tour=tour_data,
                selected_schedules=selected_schedules,
                current_photos=current_photos,
            )

        try:
            conn.execute(
                """
                UPDATE tours
                SET title = ?, theme = ?, meeting_point = ?, duration_mins = ?,
                    language = ?, max_participants = ?, description = ?
                WHERE id = ?
                """,
                (
                    data["title"],
                    data["theme"],
                    data["meeting_point"],
                    data["duration_mins"],
                    data["language"],
                    data["max_participants"],
                    data["description"],
                    id,
                ),
            )
            conn.execute("DELETE FROM tour_schedule WHERE tour_id = ?", (id,))
            conn.execute("DELETE FROM tour_stops WHERE tour_id = ?", (id,))
            for schedule in data["schedules"]:
                conn.execute(
                    "INSERT INTO tour_schedule (tour_id, weekday, start_time) VALUES (?, ?, ?)",
                    (id, schedule["weekday"], schedule["start_time"]),
                )
            for position, stop in enumerate(data["stops"], start=1):
                conn.execute(
                    "INSERT INTO tour_stops (tour_id, name, position) VALUES (?, ?, ?)",
                    (id, stop, position),
                )
            for photo_id in remove_photo_ids:
                conn.execute(
                    "DELETE FROM tour_photos WHERE tour_id = ? AND id = ?",
                    (id, photo_id),
                )
            if photo_files:
                row = conn.execute(
                    "SELECT COALESCE(MAX(position), 0) AS max_position FROM tour_photos WHERE tour_id = ?",
                    (id,),
                ).fetchone()
                start_position = row["max_position"] + 1
                for offset, file in enumerate(photo_files):
                    position = start_position + offset
                    path = save_uploaded_file(file, f"tour-{id}-{position}")
                    conn.execute(
                        "INSERT INTO tour_photos (tour_id, path, position) VALUES (?, ?, ?)",
                        (id, path, position),
                    )
            if remove_photo_ids or photo_files:
                reindex_tour_photos(conn, id)
            conn.commit()
            for path in removed_photo_paths:
```

Lines 893-979

```
                delete_uploaded_file(path)
            flash("Tour updated.", "success")
            return redirect(url_for("tour_detail", id=id))
        except ValueError as exc:
            conn.rollback()
            flash(str(exc), "danger")
        finally:
            conn.close()
    else:
        conn.close()

    return render_template(
        "create_tour.html",
        tour=tour_data,
        selected_schedules=selected_schedules,
        current_photos=current_photos,
    )


# Tour actions

@app.route("/tour/<int:id>")
def tour_detail(id):
    conn = get_db_connection()
    tour_row = get_tour_row(conn, id)
    if not tour_row:
        conn.close()
        abort(404)

    tour = enrich_tour(conn, tour_row)
    stops = get_stops(conn, id)
    photos = get_photos(conn, id)
    dates = upcoming_dates(conn, tour)
    comments = conn.execute(
        """
        SELECT comments.*, users.first_name, users.last_name, users.role
        FROM comments
        JOIN users ON users.id = comments.user_id
        WHERE comments.tour_id = ?
        ORDER BY comments.id DESC
        """,
        (id,),
    ).fetchall()
    conn.close()

    return render_template(
        "tour_detail.html",
        tour=tour,
        stops=stops,
        photos=photos,
        upcoming_dates=dates,
        comments=comments,
    )


@app.route("/tour/<int:id>/like", methods=["POST"])
def toggle_like(id):
    if not current_user.is_authenticated:
        flash("Sign in to like this tour.", "warning")
        return redirect(url_for("login", next=url_for("tour_detail", id=id)))

    conn = get_db_connection()
    if not get_tour_row(conn, id):
        conn.close()
        abort(404)

    existing = conn.execute(
        "SELECT id FROM tour_likes WHERE tour_id = ? AND user_id = ?",
        (id, current_user.id),
    ).fetchone()
    if existing:
        conn.execute("DELETE FROM tour_likes WHERE id = ?", (existing["id"],))
        flash("Like removed.", "success")
    else:
        conn.execute(
            "INSERT INTO tour_likes (tour_id, user_id) VALUES (?, ?)",
            (id, current_user.id),
        )
        flash("Like added.", "success")
    conn.commit()
    conn.close()
    return redirect(url_for("tour_detail", id=id))


@app.route("/tour/<int:id>/book", methods=["POST"])
def book_tour(id):
    if not current_user.is_authenticated:
```

Lines 980-1053

```
        flash("Sign in as a participant to book.", "warning")
        return redirect(url_for("login", next=url_for("tour_detail", id=id)))
    if current_user.role != "participant":
        flash("Guides cannot book with a guide account.", "warning")
        return redirect(url_for("tour_detail", id=id))

    conn = get_db_connection()
    tour = get_tour_row(conn, id)
    if not tour:
        conn.close()
        abort(404)

    try:
        tour_date = parse_iso_date(request.form.get("tour_date"))
        if tour_date < dt.date.today():
            raise ValueError("You cannot book a past date.")
        schedule = get_schedule_for_date(conn, id, tour_date)
        if not schedule:
            raise ValueError("The selected date is not part of this tour schedule.")
        if tour_datetime(tour_date, schedule["start_time"]) <= dt.datetime.now():
            raise ValueError("This tour date has already started or ended.")

        num_people = parse_positive_int(request.form.get("num_people"), "Number of participants", 1, 4)
        names = split_names(request.form.get("extra_names"))
        expected_extra = num_people - 1
        if len(names) != expected_extra:
            raise ValueError(
                f"Enter exactly {expected_extra} guest full name(s) for this booking."
            )
        invalid_names = [name for name in names if not has_first_and_last_name(name)]
        if invalid_names:
            raise ValueError(
                "Invalid guest name format: enter first name and last name for each guest."
            )
        seats_left = available_places(conn, tour, tour_date)
        if num_people > seats_left:
            raise ValueError(f"Not enough seats: {seats_left} left.")

        overlap = participant_has_overlap(
            conn,
            current_user.id,
            tour_date,
            schedule["start_time"],
            tour["duration_mins"],
        )
        if overlap:
            raise ValueError(
                f"You already have an overlapping booking: {overlap['title']} "
                f"from {time_range_label(overlap['start_time'], overlap['duration_mins'])}."
            )

        existing = conn.execute(
            """
            SELECT * FROM reservations
            WHERE user_id = ? AND tour_id = ? AND tour_date = ?
            """,
            (current_user.id, id, tour_date.isoformat()),
        ).fetchone()
        if existing and existing["status"] == "booked":
            raise ValueError("You have already booked this tour on this date.")
        if existing and existing["status"] == "cancelled":
            conn.execute(
                """
                UPDATE reservations
                SET num_people = ?, extra_names = ?, status = 'booked', cancelled_at = NULL
                WHERE id = ?
                """,
                (num_people, ", ".join(names), existing["id"]),
            )
        else:
            conn.execute(
                """
                INSERT INTO reservations (user_id, tour_id, tour_date, num_people, extra_names)
                VALUES (?, ?, ?, ?, ?)
```

Lines 1054-1134

```
                """,
                (current_user.id, id, tour_date.isoformat(), num_people, ", ".join(names)),
            )
        conn.commit()
        flash("Booking confirmed.", "success")
    except ValueError as exc:
        flash(str(exc), "danger")
    except IntegrityError:
        flash("A booking already exists for this date.", "danger")
    finally:
        conn.close()
    return redirect(url_for("tour_detail", id=id))


@app.route("/tour/<int:id>/comment", methods=["POST"])
def add_comment(id):
    if not current_user.is_authenticated:
        flash("Sign in to comment.", "warning")
        return redirect(url_for("login", next=url_for("tour_detail", id=id)))

    conn = get_db_connection()
    if not get_tour_row(conn, id):
        conn.close()
        abort(404)
    text = (request.form.get("text") or "").strip()
    if len(text) < 2:
        flash("The comment is too short.", "danger")
    elif len(text) > 600:
        flash("The comment cannot exceed 600 characters.", "danger")
    else:
        conn.execute(
            """
            INSERT INTO comments (tour_id, user_id, publication_date, text)
            VALUES (?, ?, ?, ?)
            """,
            (id, current_user.id, dt.date.today().isoformat(), text),
        )
        conn.commit()
        flash("Comment posted.", "success")
    conn.close()
    return redirect(url_for("tour_detail", id=id))


# Profiles

@app.route("/profile")
@login_required
def profile():
    if current_user.role == "guide":
        return redirect(url_for("guide_profile"))
    return redirect(url_for("participant_profile"))


@app.route("/participant/profile")
@login_required
def participant_profile():
    denied = require_role("participant")
    if denied:
        return denied

    conn = get_db_connection()
    rows = conn.execute(
        """
        SELECT reservations.*, tours.title, tours.meeting_point, tours.duration_mins,
               tours.language, users.first_name AS guide_first_name,
               users.last_name AS guide_last_name
        FROM reservations
        JOIN tours ON tours.id = reservations.tour_id
        JOIN users ON users.id = tours.guide_id
        WHERE reservations.user_id = ?
        ORDER BY reservations.tour_date DESC, reservations.id DESC
        """,
        (current_user.id,),
    ).fetchall()

    reservations = []
    now = dt.datetime.now()
    for row in rows:
        item = dict(row)
        tour_date = parse_iso_date(row["tour_date"])
        schedule = get_schedule_for_date(conn, row["tour_id"], tour_date)
```

Lines 1135-1214

```
        item["start_time"] = schedule["start_time"] if schedule else "-"
        item["can_cancel"] = False
        if row["status"] == "booked" and schedule:
            start_dt = tour_datetime(tour_date, schedule["start_time"])
            item["can_cancel"] = start_dt - now >= dt.timedelta(hours=24)
        reservations.append(item)
    conn.close()
    return render_template("participant_profile.html", reservations=reservations)


@app.route("/reservation/<int:reservation_id>/cancel", methods=["POST"])
@login_required
def cancel_reservation(reservation_id):
    denied = require_role("participant")
    if denied:
        return denied

    conn = get_db_connection()
    reservation = conn.execute(
        "SELECT * FROM reservations WHERE id = ? AND user_id = ?",
        (reservation_id, current_user.id),
    ).fetchone()
    if not reservation:
        conn.close()
        abort(404)
    if reservation["status"] != "booked":
        flash("This booking has already been cancelled.", "warning")
        conn.close()
        return redirect(url_for("participant_profile"))

    tour_date = parse_iso_date(reservation["tour_date"])
    schedule = get_schedule_for_date(conn, reservation["tour_id"], tour_date)
    if not schedule or tour_datetime(tour_date, schedule["start_time"]) - dt.datetime.now() < dt.timedelta(hours=24):
        flash("You can cancel only at least 24 hours before the tour starts.", "danger")
        conn.close()
        return redirect(url_for("participant_profile"))

    conn.execute(
        """
        UPDATE reservations
        SET status = 'cancelled', cancelled_at = CURRENT_TIMESTAMP
        WHERE id = ?
        """,
        (reservation_id,),
    )
    conn.commit()
    conn.close()
    flash("Booking cancelled.", "success")
    return redirect(url_for("participant_profile"))


@app.route("/guide/profile")
@login_required
def guide_profile():
    denied = require_role("guide")
    if denied:
        return denied

    conn = get_db_connection()
    rows = conn.execute(
        """
        SELECT tours.*, users.first_name AS guide_first_name,
               users.last_name AS guide_last_name
        FROM tours
        JOIN users ON users.id = tours.guide_id
        WHERE guide_id = ?
        ORDER BY tours.created_at DESC, tours.id DESC
        """,
        (current_user.id,),
    ).fetchall()
    tours = []
    now = dt.datetime.now()
    for row in rows:
        tour = enrich_tour(conn, row)
        groups = conn.execute(
            """
            SELECT tour_date, COUNT(*) AS reservation_count,
                   COALESCE(SUM(num_people), 0) AS people_count
            FROM reservations
            WHERE tour_id = ? AND status = 'booked'
```

Lines 1215-1291

```
            GROUP BY tour_date
            ORDER BY tour_date
            """,
            (tour["id"],),
        ).fetchall()
        tour["reservation_dates"] = []
        for group in groups:
            tour_date = parse_iso_date(group["tour_date"])
            schedule = get_schedule_for_date(conn, tour["id"], tour_date)
            start_time = schedule["start_time"] if schedule else "-"
            start_dt = tour_datetime(tour_date, start_time) if schedule else None
            report = conn.execute(
                "SELECT * FROM tour_reports WHERE tour_id = ? AND tour_date = ?",
                (tour["id"], group["tour_date"]),
            ).fetchone()
            reservations = conn.execute(
                """
                SELECT reservations.*, users.first_name, users.last_name, users.email
                FROM reservations
                JOIN users ON users.id = reservations.user_id
                WHERE reservations.tour_id = ?
                  AND reservations.tour_date = ?
                  AND reservations.status = 'booked'
                ORDER BY reservations.id
                """,
                (tour["id"], group["tour_date"]),
            ).fetchall()
            tour["reservation_dates"].append(
                {
                    "date": group["tour_date"],
                    "start_time": start_time,
                    "reservation_count": group["reservation_count"],
                    "people_count": group["people_count"],
                    "reservations": reservations,
                    "report": report,
                    "can_report": bool(start_dt and start_dt < now and not report),
                }
            )
        tours.append(tour)
    conn.close()
    return render_template("guide_profile.html", tours=tours)


# Reports

@app.route("/guide/report/<int:tour_id>/<tour_date>", methods=["POST"])
@login_required
def submit_report(tour_id, tour_date):
    denied = require_role("guide")
    if denied:
        return denied

    conn = get_db_connection()
    tour = get_tour_row(conn, tour_id)
    if not tour:
        conn.close()
        abort(404)
    if tour["guide_id"] != int(current_user.id):
        conn.close()
        abort(403)

    try:
        parsed_date = parse_iso_date(tour_date)
        schedule = get_schedule_for_date(conn, tour_id, parsed_date)
        if not schedule:
            raise ValueError("This date does not belong to the tour schedule.")
        if tour_datetime(parsed_date, schedule["start_time"]) >= dt.datetime.now():
            raise ValueError("The report can be submitted only after the tour has taken place.")

        expected = active_reserved_places(conn, tour_id, parsed_date)
        if expected <= 0:
            raise ValueError("There are no bookings for this date.")
        existing_report = conn.execute(
            "SELECT id FROM tour_reports WHERE tour_id = ? AND tour_date = ?",
            (tour_id, tour_date),
        ).fetchone()
        if existing_report:
```

Lines 1292-1305

```
            raise ValueError("The report for this date has already been submitted.")
        actual = parse_positive_int(
            request.form.get("actual_participants"),
            "Actual participants",
            0,
            expected,
        )
        file = request.files.get("evidence_photo")
        if not validate_file(file):
            raise ValueError("Upload a valid evidence photo.")
        photo_path = save_uploaded_file(file, f"report-{tour_id}-{tour_date}")

        conn.execute(
            """
```

Lines 1306-1321

```
            INSERT INTO tour_reports (tour_id, tour_date, actual_participants, evidence_photo)
            VALUES (?, ?, ?, ?)
            """,
            (tour_id, tour_date, actual, photo_path),
        )
        conn.commit()
        flash("Report saved.", "success")
    except ValueError as exc:
        flash(str(exc), "danger")
    finally:
        conn.close()
    return redirect(url_for("guide_profile"))


if __name__ == "__main__":
    app.run(host="0.0.0.0", debug=True)
```